INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Adaptive Web Penetration Testing with Q-Learning and Deep Q-Networks on OWASP Juice Shop

Kevin Imanuel Setiawan

Bachelor's Program in Information Technology, Telkom University, Surabaya, Indonesia

* Corresponding author: kevinit@student.telkomuniversity.ac.id

Abstract: This study compares tabular Q-learning (QL) and a deep Q-network (DQN) for adaptive web penetration testing on OWASP Juice Shop. Both agents operated for 1,585 episodes over 120 predefined endpoint-payload actions. Candidate findings passed control-differential verification, while cross-site scripting candidates also required browser execution. QL produced 14 verified action-level findings and DQN produced 13, across five unique endpoint-class locations each. Both agents found their first verified action in episode 1; QL reached its final observed count by episode 787 and DQN by episode 443. A separate detector benchmark yielded 13 true positives, no false positives, and one false negative among labeled actions; 46 actions remained unknown. A scoped two-endpoint comparison recorded 10 QL findings and one relevant ZAP alert, but the request procedures were not equivalent. The results demonstrate feasibility on this target and action set, not generalization to other applications.

Keywords: Web penetration testing, Reinforcement learning, Q-learning, Deep Q-network, OWASP Juice Shop, OWASP ZAP

1. Introduction

Web applications expose multiple input points and behaviors, so penetration testers must choose requests under limited time and request budgets. Reinforcement learning offers a way to learn action preferences from observed responses. Q-learning (QL) estimates action values in a table, whereas a deep Q-network (DQN) approximates values with a neural network [1]–[3]. Prior research has explored reinforcement learning for penetration testing and capture-the-flag environments, while emphasizing trade-offs between model-free learning and prior knowledge [4].

Prior penetration-testing and capture-the-flag studies highlight trade-offs between model-free exploration and prior knowledge [4]. A predefined endpoint and payload library improves repeatability but narrows the search. Here, the policy selects among human-curated endpoints, surfaces, and payloads; it does not crawl, discover routes, or generate new payloads.

The Agent Web Model and cyber-terrain-aware attack graphs emphasize that representation and operational constraints shape RL behavior [5], [6]. We disclose these boundaries and verify candidates before counting findings.

This study compares QL and DQN on OWASP Juice Shop, a deliberately vulnerable security-training application [7], and reports a scoped search/FTP comparison with ZAP. Candidates require control-differential verification; XSS also requires browser execution. Results apply to this target and action set.

We compare verified findings, categories, endpoint activity, reward, and discovery timing under a fixed episode budget, then describe what the non-equivalent ZAP workflows allow us to infer. No statistical significance or generalization is claimed.

2. Materials and Methods

2.1 Test Environment and Action Space

The target was OWASP Juice Shop version 20.2.0 running locally. Eight configured endpoints and 15 payloads formed 120 endpoint-payload actions: six SQL injection (SQLi), four cross-site scripting (XSS), and five fuzzing probes. Endpoint surface labels and payloads were manually curated from knowledge of the application. This human prior narrows the search space, so the system is not fully autonomous. The state comprises HTTP status, response length, error, reflection and vulnerability indicators, endpoint index, response-time category, and content type. DQN receives a 28-dimensional one-hot encoding.

The eight endpoint configurations cover product search, file access, product reviews, order tracking, user login, feedback, and challenge retrieval. GET probes are placed in the configured query or path; login, review updates, and feedback use structured request bodies. The feedback operation also obtains a captcha through the application endpoint and supplies required fields. These endpoint-specific request formats reduce the chance that malformed traffic is mistaken for evidence of security, while keeping the experiment within the implemented API flows.

Table 1. Configured endpoints and input surfaces.

Index	Method and path	Surface
0	GET /rest/products/search?q=	SQL
1	GET /ftp/	File
2	GET /rest/products/1/reviews	Plain
3	GET /rest/track-order/	Plain
4	POST /rest/user/login	SQL
5	PUT /rest/products/1/reviews	Plain
6	POST /api/Feedbacks	Plain
7	GET /api/Challenges	Plain

Action a maps to endpoint floor(a/15) and payload a modulo 15. The fixed library is selected by the policy; routes and payloads are not generated at runtime.

The state tuple records status, response-length category, error, reflection and vulnerability indicators, endpoint index, response-time category, and content type. It is an engineered summary, not a full application model. DQN one-hot encodes status (5), length (4), endpoint (8), response time (3), and content type (5), plus three scalar flags, for 28 inputs; QL uses the tuple as its key.

Table 2. State variables and encodings.

Field	Encoding	Role
HTTP status	5 categories	Response class
Response length	4 categories	Size category
Error	Scalar flag	Request/server error evidence
Reflection	Scalar flag	Input-reflection evidence
Vulnerability	Scalar indicator	Analyzer classification
Endpoint	8 categories	Request location
Response time	3 categories	Latency category
Content type	5 categories	Response format

2.1.1 HTTP Response Preprocessing and State Construction

The analyzer lowercases decoded response text and measures its character count, not network bytes. Missing responses become error states with zero status and length and no vulnerability label.

The analyzer extracts status, latency and content-type classes, database-error patterns, payload-family matches, reflection, and baseline change. Partial reflection requires at least half of meaningful payload tokens (each at least four characters) in the response.

Before training, each endpoint receives one benign "hello" probe. A length change is significant when $|L - B_e|/B_e \geq 0.15$. Categories are below 1,500, 1,500–<2,500, 2,500–<3,500, and 3,500 or more characters.

Status has five classes (2xx, 3xx, 4xx, 5xx, error), response time three (<0.5 s, 0.5–<2.0 s, ≥2.0 s), and content type five (JSON, HTML, text, XML, other). The system engineers these features rather than vectorizing response text.

2.2 Agent Configuration and Training Protocol

Each agent used 1,585 episodes and random seed 42. QL used learning rate 0.1, discount factor 0.9, epsilon from 1.0 to 0.1 with decay 0.90, and optimistic initial values of 5. DQN used a 64-unit hidden layer, learning rate 0.001, batch size 32, replay capacity 10,000, target update interval 10, and epsilon decay 0.9965. Exploration schedules differed to avoid DQN reaching its exploration floor too early. There was no early stopping. Only verified vulnerable actions were masked; previously attempted or failed actions remained available. A benign hello probe established a separate response baseline for each endpoint. Rewards were +50 for verified findings, +10 for anomalies or inconclusive candidates, −1 for normal responses, and −5 for errors, with a configured response-length bonus of 0, 0, 5, or 10.

Table 3. Agent hyperparameters.

Parameter	Q-learning	DQN
Episode budget	1,585	1,585
Learning rate	0.1	0.001
Discount factor	0.9	0.9
Exploration	ε: 1.0 to 0.1; decay 0.90	ε: 1.0 to 0.1; decay 0.9965
Value initialization	5.0	Neural network
Hidden units	—	64
Batch / replay	—	32 / 10,000
Target update	—	Every 10 updates
Random seed	42	42

The two methods use the same environment and observations but represent action values differently. QL updates a discrete table according to $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, where s and s' denote the current and next states, a is the selected action, r is its reward, α is the learning rate, and γ discounts future value [1], [2]. An epsilon-greedy policy samples an action for exploration with probability ε and otherwise selects a high-valued action. Optimistic initialization assigns unvisited QL values a starting estimate of 5.0.

DQN learns from replay transitions with a target network. Its gradual 0.9965 epsilon decay preserves exploration under random initial weights; hyperparameters follow each algorithm rather than being numerically identical.

Reward is computed after candidate verification. The base outcomes and length-bonus categories are summarized in Table 4.

Verified candidates earn +50, anomalies or inconclusive candidates +10, and normal responses −1; connection errors earn −5 without a length bonus. Completed responses also receive the category bonus in Table 4.

With bonus vector [0, 0, 5, 10], examples are +60 for a verified category-3 finding, +15 for an inconclusive category-2 response, +9 for a normal category-3 response, and −5 for a request error. A large response can raise reward without proving a vulnerability.

Cumulative reward sums findings, anomalies, normal responses, errors, and length bonuses; it is not a coverage metric. We report it alongside verified counts, hit rate, discovery episode, and request cost.

Table 4. Reward components.

Outcome	Reward	Purpose
Verified vulnerability	+50	Positive signal after verification
Anomaly/inconclusive	+10	Signal without verified-vulnerability claim
Normal response	−1	Small negative signal
Error response	−5	Penalty for failed request
Length bonus	0, 0, +5, +10	Category-based additional reward

Figure 1 summarizes the loop: collect a benign baseline per endpoint, select and send an action, analyze its response, verify candidates, assign reward, and update the agent. This keeps detection, verification, and learning in sequence.

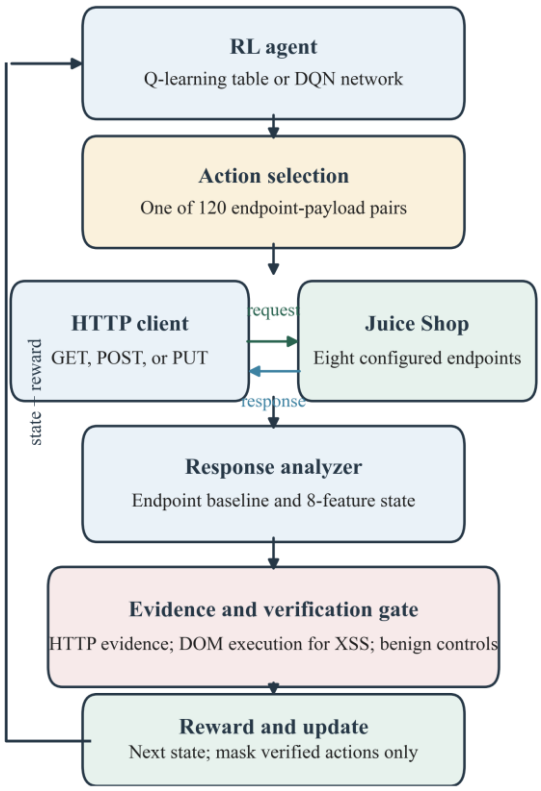


Fig. 1. System workflow for reinforcement-learning-based web testing.

Only verified vulnerable actions are masked. Failed, normal, previously attempted, and rejected actions remain selectable; both agents complete all 1,585 episodes, including after their final discovery.

2.3 Finding Verification and Detector Benchmark

Each HTTP candidate was compared with a benign control and, when meaningful, a neutralized-payload control; neither could reproduce the same vulnerability class. Connection errors failed

verification. XSS required an instrumented canary to execute in headless Chromium on the SPA route, using warmup and bounded polling. The separate 120-action benchmark left 46 unknown labels outside binary metrics and partly shared verification evidence.

Detection was surface-gated: SQLi required database-error or significant baseline-change evidence; file disclosure required recognizable content on the file surface or configured extension-bypass evidence; fuzzing required an echoed probe in a server error. Generic errors and reflection alone were excluded.

Neutralized controls test whether benign or less exploit-like input reproduces evidence; payloads that cannot be meaningfully neutralized use only the benign control. Inconclusive cases earn anomaly reward. Browser execution distinguishes reflected data from executing XSS.

2.4 ZAP Comparison and Metrics

QL and ZAP were compared on their two shared endpoints, product search and FTP [8]. We report findings, hit rate, requests per finding, first and final discovery episode, and unique endpoint-class locations. Results are in-sample; shared scope does not equalize scan strategy, verification, or request accounting.

2.4.1 ZAP Architecture and Controlled Scan Flow

ZAP provides a local proxy, Core API, spider, active scanner, passive scanner, and alert store. The controller reads HTTP history and alerts through the API; alerts include risk, URL, parameter, evidence, and attack [9]–[12].

ZAP 2.17 ran at localhost:8080 against Juice Shop at localhost:3000. For search and /ftp/, the Python controller established a message baseline, accessed each URL, ran the spider (maxChildren=2) to completion, then ran a nonrecursive active scan. No authenticated user or spider-wide target was configured. The report counted 590 messages (22 URL/spider and 568 active-scan) and 33 raw alerts; one High/Medium SQLi alert met the study filter.

The controller estimated phase request counts from message-history deltas, then filtered alerts by class (SQLi, XSS, path traversal) and High/Medium risk. This is the study's comparison rule; excluded alerts may remain valid findings under other criteria.

Figure 2 separates the controller and alert filter from ZAP's spider and active scanner. Although passive scanning can analyze recorded traffic, this script did not run a separate passive-scan job; active scan generated attack probes.

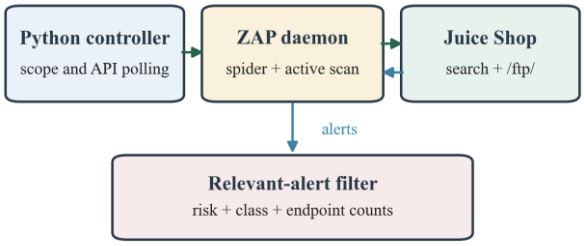


Fig. 2. Architecture and request flow of the controlled ZAP scan.

A finding count and a unique vulnerable location are also different measures. The unique-location measure counts distinct endpoint-index and vulnerability-class pairs. Several accepted payload-level findings at the same endpoint and class can contribute multiple findings but one location. This distinction prevents payload variants from being presented as separate vulnerable application locations.

The 1,585 episodes are the learning budget for each agent. The experiment also records baseline probes, verification probes, and total requests including verification. Episode count, selected payload-request count, and end-to-end HTTP transaction count therefore represent different quantities. Request-per-finding ratios must be read with their counting rule, particularly because control probes are issued only after candidates are detected.

3. Results and Discussion

3.1 QL and DQN Training Results

QL produced 14 verified action-level findings and DQN produced 13 (Table 5). Both found a vulnerability in episode 1. QL reached its final observed count by episode 787, whereas DQN did so by episode 443. These values describe accumulation during the recorded run and do not establish convergence or unseen-target performance. QL had more findings, while DQN accumulated a larger total reward, which reflects reward across all episodes rather than finding count alone.

Table 5. Training outcomes.

Measure	QL	DQN
Episodes	1,585	1,585
Verified findings	14	13
Categories	6 SQLi; 5 XSS; 3 fuzz/file	6 SQLi; 5 XSS; 2 fuzz/file
Hit rate	0.88%	0.82%
Requests/finding	113.21	121.92
First finding	Episode 1	Episode 1
Final count reached	Episode 787	Episode 443
Unique locations	5	5
Elapsed time	143.64 s	145.81 s
Total reward	3,783	13,033
Total transactions incl. baseline/verification	1,640	1,637

The category counts show six SQLi and five XSS action-level findings for each agent, while QL reported one additional fuzzing/file finding (Fig. 4).

These are action-level findings, not 14 or 13 independently confirmed production vulnerabilities. Multiple payloads can map to one endpoint-class location; each run covered five distinct locations.

Both agents found their first action in episode 1. DQN reached its final count by episode 443, before QL at 787; these are milestones in one run, not evidence of convergence. Figure 3 shows cumulative unique verified actions; repeated seeds are needed to assess stability.

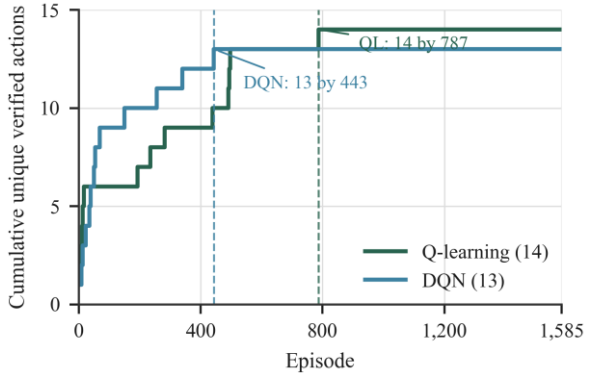


Fig. 3. Cumulative unique verified-action discoveries by episode.

The cumulative reward should not be interpreted as a direct vulnerability-coverage metric. It aggregates rewards for every episode, including anomalies, normal responses, errors, and length bonuses. DQN can therefore earn a higher total reward even while reporting fewer verified findings. Future comparisons should report reward and verified-finding curves together and summarize variation over repeated seeds.

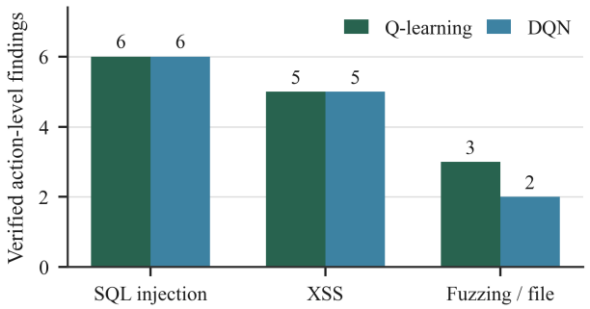


Fig. 4. Verified action-level findings by category.

3.2 QL Endpoint Activity

Product search accounted for most QL requests and findings. The FTP surface produced three findings from 41 requests; track-order produced three; login produced one. Other configured endpoints produced no verified findings in this run. The distribution reflects the curated action space and should not be interpreted as a complete vulnerability inventory.

The endpoint request counts sum to the 1,585 episode actions. Product search accounts for 1,173 requests, or roughly three quarters of the budget, while track-order accounts for 245. The other six endpoints together account for 149 requests. The learned policy therefore concentrated its actions at search and track-order, especially search. This describes policy allocation in the run; it does not demonstrate that the remaining endpoints were exhaustively tested outside the configured payload set.

The endpoint finding counts sum to 14: seven for search, three for FTP, three for track-order, and one for login. The report separately gives five unique vulnerable locations because multiple accepted payload findings can correspond to the same endpoint-class pair. Alert consolidation rules vary between tools, so any cross-tool comparison must state whether it counts payload-level findings, unique locations, or consolidated alerts.

The higher DQN reward despite fewer findings follows from the reward design: it includes normal responses, errors, anomalies, and response-length bonuses throughout the run. Reward is an optimization signal, not a substitute for coverage.

Table 6. QL requests and verified findings by endpoint.

Endpoint	Requests	Findings
Product search	1,173	7
FTP surface	41	3

Product reviews (GET)	39	0
Track order	245	3
User login	24	1
Product reviews (PUT)	21	0
Feedback	22	0
Challenges	20	0
Total	1,585	14

3.3 Detector Benchmark

The labeled benchmark contained 13 true positives, no false positives, one false negative, and 60 true negatives; 46 actions remained unknown. Precision was 1.000, recall 0.929, F1 0.963, and specificity 1.000. The false negative was the SVG XSS action on the track-order route. Zero false positives applies only to the 60 labeled negatives and does not establish perfect performance on untested applications or payloads.

Precision is TP/(TP+FP), recall is TP/(TP+FN), F1 is the harmonic mean of precision and recall, and specificity is TN/(TN+FP). These values are computed only for labeled actions. The 46 unknown actions are not assumed safe and are excluded from the negative class; recoding them as negatives would inflate specificity without evidence. The one false negative, an SVG XSS action on the track-order route, also shows that the configured detector and verification path did not recover every curated positive.

No false positives among 60 labeled negative actions is useful evidence that the gates suppressed several noisy response patterns in this benchmark. It is not a guarantee of zero false positives in future tests: the labeled set is finite, labels are curated, and the oracle partly shares the detector's evidence mechanisms. A stronger independent assessment would have blinded reviewers adjudicate browser and server evidence without seeing detector output or agent identity.

Table 7. Detector performance on labeled actions.

Metric	Value
True positives	13
False positives	0
False negatives	1
True negatives	60
Unknown actions	46
Precision	1.000
Recall	0.929
F1-score	0.963
Specificity	1.000

3.4 Scoped Comparison with OWASP ZAP

On search and FTP, QL made 1,214 payload requests for 10 findings (121.40 per finding); ZAP made 590 requests for one relevant finding (590 per finding). Of ZAP's 33 raw alerts, one High/Medium SQLi alert met the filter; none for XSS or path traversal did. The 4.86 rate ratio is descriptive because workflows, verification, and request accounting differ.

Restricting both tools to product search and FTP makes the target scope more comparable than comparing QL's eight-endpoint run with a broader scan. Still, QL selects individual payload actions through its policy, while ZAP follows its own spidering and active-scan workflow. They can issue different request types and volumes, apply different payloads, and use different alert-consolidation and acceptance criteria. QL verification probes and the ZAP workflow also contribute differently to the request totals.

The ratio does not normalize scan duration, payload complexity, severity, or review effort. A stronger study would align the endpoint allowlist, authentication, request and time budgets, payload families, counting rules, and independent manual review.

Table 8. Scoped comparison on two shared endpoints.

Method	Requests	Relevant findings	Requests/finding
QL selected-payload subset	1,214	10	121.40
Controlled ZAP scan	590	1	590.00

3.5 Interpretation of the QL–DQN Difference

The finite action set contains 120 choices and the state representation is compact and engineered. Tabular QL is consequently a reasonable baseline because values can be stored for discrete observed states without function approximation. DQN may

be useful when state representations grow or share structure, but this experiment does not establish that advantage. The finding totals and reward totals are outcomes of one run under different exploration schedules; they should be treated as an implementation comparison rather than a general ranking of learning algorithms.

3.6 Threats to Validity

Construct validity is limited by the operational definition of a finding: a candidate must match the analyzer rules and pass its verification gate. The evaluation does not assign severity or measure business impact. Internal validity is limited by a single random seed per agent and algorithm-specific hyperparameters. The observed differences may reflect trajectory randomness and configuration choices as well as the learning algorithm.

External validity is limited to one version of one intentionally vulnerable application, one local deployment, and one curated payload library. Endpoint surfaces and payload choices encode human knowledge. The test does not cover changing production state, real authentication workflows, rate limits, network variability, or a broad range of web frameworks. A fixed seed improves reproducibility but cannot estimate run-to-run variation.

Comparison validity is limited because ZAP and the RL agents use different scan workflows and request-count rules. Equal endpoint scope alone does not ensure equal opportunities to find vulnerabilities. Consolidation rules and evidence thresholds also affect the finding numerator. These limitations constrain the interpretation of the 4.86 ratio and preclude a general efficiency claim.

3.7 Follow-up Experiments

Future work should run each method across multiple seeds and report mean, spread, confidence intervals, and per-seed discovery curves. A held-out application or endpoint set would test transfer, provided the held-out labels are not used to design the action space or detection rules. Ablations could remove surface gating, browser verification, or optimistic initialization to estimate how each design choice affects discovery and false positives.

For a stronger ZAP comparison, both systems should use a published endpoint allowlist and aligned authentication state. Request budgets and wall-clock budgets should be reported separately, since equalizing only one can favor a system with more expensive verification. Candidate findings should be deduplicated by endpoint, vulnerability class, and evidence, then manually reviewed by an assessor blinded to tool identity. Coverage, request cost, time-to-first-verified-finding, and false-positive review burden can then be compared under a shared protocol.

4. Conclusion

Under 1,585 episodes on OWASP Juice Shop, QL obtained 14 verified action-level findings and DQN obtained 13. DQN reached its final observed finding count earlier, while accumulating greater total reward. The detector benchmark agreed strongly with curated labels, subject to partial dependence on shared verification evidence. The scoped ZAP comparison favored QL in requests per finding, but procedural differences make that result descriptive. Repeated and held-out evaluations are required before drawing broader conclusions.

Conflict of Interest

The author declares that there is no conflict of interest.

References

[1] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, Vol. 8, No. 3–4, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, Cambridge, MA, 2018.

[3] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, Vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[4] F. M. Zennaro and L. Erdodi, “Modeling penetration testing with reinforcement learning using capture-the-flag challenges: Trade-offs between model-free learning and a priori knowledge,” arXiv:2005.12632, 2020.

[5] L. Erdodi and F. M. Zennaro, “The Agent Web Model: Modelling web hacking for reinforcement learning,” arXiv:2009.11274, 2020.

[6] R. Gangupantulu, T. Cody, P. Park, A. Rahman, L. Eisenbeiser, D. Radke, and R. Clark, “Using cyber terrain in reinforcement learning for penetration testing,” arXiv:2108.07124, 2021.

[7] OWASP Foundation, “OWASP Juice Shop,” [Online]. Available: https://owasp.org/projects/juice-shop?tab=overview. Accessed: Oct. 12, 2026.

[8] OWASP ZAP, “Automation Framework,” [Online]. Available: https://www.zaproxy.org/docs/automate/automation-framework/. Accessed: Oct. 12, 2026.

[9] OWASP ZAP, “Getting Started,” [Online]. Available: https://www.zaproxy.org/docs/desktop/start/. Accessed: Oct. 12, 2026.

[10] OWASP ZAP, “API Reference,” [Online]. Available: https://www.zaproxy.org/docs/api/. Accessed: Oct. 12, 2026.

[11] OWASP ZAP, “Passive Scanner Add-on,” [Online]. Available: https://www.zaproxy.org/docs/desktop/addons/passive-scanner/. Accessed: Oct. 12, 2026.

[12] OWASP ZAP, “Automation Framework: Active Scan Job,” [Online]. Available: https://www.zaproxy.org/docs/desktop/addons/automation-framework/job-ascan/. Accessed: Oct. 12, 2026.